

DSA Interview Revision

Complete Notes with C++ Code

LeetCode Top Interview 150 | Blind 75 | Striver's SDE Sheet

Language: C++ | All key patterns, problems & approaches

Table of Contents

1. Arrays & Hashing
2. Two Pointers
3. Sliding Window
4. Binary Search
5. Sorting Algorithms
6. Linked Lists
7. Stacks & Queues
8. Trees (Binary Tree & BST)
9. Heaps / Priority Queues
10. Tries
11. Graphs
12. Dynamic Programming
13. Greedy Algorithms
14. Backtracking & Recursion
15. Bit Manipulation
16. Intervals
17. Math & Matrix
18. Design Problems (LRU Cache etc.)
19. Complexity Cheat Sheet & Pattern Map

1. Arrays & Hashing

Core Concepts

- Array: Contiguous memory, $O(1)$ index access, $O(n)$ insert/delete. Use `std::vector` in C++.
- Hash Map: `unordered_map<K,V>` gives $O(1)$ avg lookup/insert. Use for frequency counting, two-sum style, grouping.
- Key patterns: Prefix sums, frequency maps, Kadane's algorithm, Dutch National Flag, sliding window.

>> Two Sum (LC #1)

Easy | Hash Map

Given array `nums` and `target`, return indices of two numbers that add up to `target`. Single pass with hash map.

```
vector<int> twoSum(vector<int>& nums, int target) {
    unordered_map<int, int> mp;
    for (int i = 0; i < nums.size(); i++) {
        int comp = target - nums[i];
        if (mp.count(comp)) return {mp[comp], i};
        mp[nums[i]] = i;
    }
    return {};
```

Time: $O(n)$ | Space: $O(n)$

>> Contains Duplicate (LC #217)

Easy | Hash Set

Return true if any value appears at least twice.

```
bool containsDuplicate(vector<int>& nums) {
    unordered_set<int> seen;
    for (int n : nums) {
        if (seen.count(n)) return true;
        seen.insert(n);
    }
    return false;
```

>> Best Time to Buy and Sell Stock (LC #121)

Easy | Greedy / One Pass

Find max profit from one buy-sell transaction. Track min price seen so far.

```
int maxProfit(vector<int>& prices) {
    int minPrice = INT_MAX, maxProfit = 0;
    for (int p : prices) {
        minPrice = min(minPrice, p);
        maxProfit = max(maxProfit, p - minPrice);
    }
    return maxProfit;
```

Time: $O(n)$ | Space: $O(1)$

>> Maximum Subarray - Kadane's (LC #53)

Medium | DP / Greedy

Find contiguous subarray with largest sum. Kadane's: keep running sum, reset if it becomes negative.

```
int maxSubArray(vector<int>& nums) {
    int curSum = nums[0], maxSum = nums[0];
    for (int i = 1; i < nums.size(); i++) {
        curSum = max(nums[i], curSum + nums[i]);
        maxSum = max(maxSum, curSum);
    }
    return maxSum;
```

>> Product of Array Except Self (LC #238)

Medium | Prefix / Suffix

Return array where `output[i]` = product of all elements except `nums[i]`. No division allowed.

```
vector<int> productExceptSelf(vector<int>& nums) {
    int n = nums.size();
    vector<int> res(n, 1);
    int prefix = 1;
    for (int i = 0; i < n; i++) {
        res[i] = prefix;
        prefix *= nums[i];
    }
    int suffix = 1;
    for (int i = n - 1; i >= 0; i--) {
        res[i] *= suffix;
        suffix *= nums[i];
    }
    return res;
}
```

Time: $O(n)$ | Space: $O(1)$ extra (output doesn't count)

>> Group Anagrams (LC #49)

Medium | Hash Map + Sorted Key

Group strings that are anagrams. Use sorted string as key.

```
vector<vector<string>> groupAnagrams(vector<string>& strs) {
    unordered_map<string, vector<string>> mp;
    for (auto& s : strs) {
        string key = s;
        sort(key.begin(), key.end());
        mp[key].push_back(s);
    }
    vector<vector<string>> res;
    for (auto& [k, v] : mp) res.push_back(v);
    return res;
}
```

>> Top K Frequent Elements (LC #347)

Medium | Hash Map + Bucket Sort

Return k most frequent elements. Bucket sort by frequency for $O(n)$.

```
vector<int> topKFrequent(vector<int>& nums, int k) {
    unordered_map<int, int> cnt;
    for (int n : nums) cnt[n]++;
    vector<vector<int>> buckets(nums.size() + 1);
    for (auto& [num, freq] : cnt) buckets[freq].push_back(num);
    vector<int> res;
    for (int i = buckets.size() - 1; i >= 0 && res.size() < k; i--)
        for (int num : buckets[i]) {
            res.push_back(num);
            if (res.size() == k) break;
        }
    return res;
}
```

>> Longest Consecutive Sequence (LC #128)

Medium | Hash Set

Find length of longest consecutive elements sequence in $O(n)$. Only start counting from sequence starts.

```
int longestConsecutive(vector<int>& nums) {
    unordered_set<int> s(nums.begin(), nums.end());
    int longest = 0;
    for (int n : s) {
        if (!s.count(n - 1)) { // start of sequence
            int len = 1;
            while (s.count(n + len)) len++;
            longest = max(longest, len);
        }
    }
    return longest;
}
```

>> Set Matrix Zeroes (LC #73)

Medium | Matrix, In-place Markers

If element is 0, set its entire row and column to 0. Use first row/col as markers for $O(1)$ extra space.

```
void setZeroes(vector<vector<int>>& matrix) {
    int m = matrix.size(), n = matrix[0].size();
    bool firstRow = false, firstCol = false;
    for (int j = 0; j < n; j++) if (matrix[0][j] == 0) firstRow = true;
    for (int i = 0; i < m; i++) if (matrix[i][0] == 0) firstCol = true;
    for (int i = 1; i < m; i++)
        for (int j = 1; j < n; j++)
            if (matrix[i][j] == 0) matrix[i][0] = matrix[0][j] = 0;
    for (int i = 1; i < m; i++)
        for (int j = 1; j < n; j++)
            if (matrix[i][0] == 0 || matrix[0][j] == 0) matrix[i][j] = 0;
    if (firstRow) for (int j = 0; j < n; j++) matrix[0][j] = 0;
    if (firstCol) for (int i = 0; i < m; i++) matrix[i][0] = 0;
}
```

>> Valid Anagram (LC #242)

Easy | Hash Map / Sorting

```
bool isAnagram(string s, string t) {
    if (s.size() != t.size()) return false;
    int cnt[26] = {};
    for (int i = 0; i < s.size(); i++) {
        cnt[s[i] - 'a']++;
        cnt[t[i] - 'a']--;
    }
    for (int c : cnt) if (c != 0) return false;
    return true;
}
```

>> Majority Element (LC #169)

Easy | Boyer-Moore Voting

Element appearing more than $n/2$ times. Boyer-Moore: keep candidate + count.

```
int majorityElement(vector<int>& nums) {
    int candidate = 0, count = 0;
    for (int n : nums) {
        if (count == 0) candidate = n;
        count += (n == candidate) ? 1 : -1;
    }
    return candidate;
}
```

2. Two Pointers

Core Concept

Use two pointers (start/end or slow/fast) to scan data efficiently. Works on sorted arrays, linked lists, palindrome checks. Reduces $O(n^2)$ brute force to $O(n)$.

>> Valid Palindrome (LC #125)

Easy | Two Pointers

```
bool isPalindrome(string s) {
    int l = 0, r = s.size() - 1;
    while (l < r) {
        while (l < r && !isalnum(s[l])) l++;
        while (l < r && !isalnum(s[r])) r--;
        if (tolower(s[l]) != tolower(s[r])) return false;
        l++; r--;
    }
    return true;
}
```

>> Two Sum II - Sorted Array (LC #167)

Medium | Two Pointers

```
vector<int> twoSum(vector<int>& nums, int target) {
    int l = 0, r = nums.size() - 1;
    while (l < r) {
        int sum = nums[l] + nums[r];
        if (sum == target) return {l + 1, r + 1};
        else if (sum < target) l++;
        else r--;
    }
    return {};
}
```

>> 3Sum (LC #15)

Medium | Sort + Two Pointers

Find all unique triplets that sum to 0. Sort, fix one element, use two pointers for rest. Skip duplicates.

```
vector<vector<int>> threeSum(vector<int>& nums) {
    sort(nums.begin(), nums.end());
    vector<vector<int>> res;
    for (int i = 0; i < (int)nums.size() - 2; i++) {
        if (i > 0 && nums[i] == nums[i-1]) continue;
        int l = i + 1, r = nums.size() - 1;
        while (l < r) {
            int sum = nums[i] + nums[l] + nums[r];
            if (sum < 0) l++;
            else if (sum > 0) r--;
            else {
                res.push_back({nums[i], nums[l], nums[r]});
                while (l < r && nums[l] == nums[l+1]) l++;
                while (l < r && nums[r] == nums[r-1]) r--;
                l++; r--;
            }
        }
    }
    return res;
}
```

>> Container With Most Water (LC #11)

Medium | Two Pointers

Find two lines that form container with most water. Move the shorter line inward.

```
int maxArea(vector<int>& height) {
    int l = 0, r = height.size() - 1, res = 0;
    while (l < r) {
        res = max(res, min(height[l], height[r]) * (r - l));
        if (height[l] < height[r]) l++;
        else r--;
    }
    return res;
}
```

>> Trapping Rain Water (LC #42)

Hard | Two Pointers

Calculate trapped water. Track left_max and right_max with two pointers. Process the smaller side.

```
int trap(vector<int>& height) {
    int l = 0, r = height.size() - 1;
    int leftMax = 0, rightMax = 0, water = 0;
    while (l < r) {
        if (height[l] < height[r]) {
            leftMax = max(leftMax, height[l]);
            water += leftMax - height[l];
            l++;
        } else {
            rightMax = max(rightMax, height[r]);
            water += rightMax - height[r];
            r--;
        }
    }
    return water;
}
```

Time: O(n) | Space: O(1)

3. Sliding Window

Core Concept

Maintain a window [l, r] that expands/shrinks. Two types: Fixed-size (both pointers move together) and Variable-size (expand r, shrink l when invalid). Typical O(n).

>> Longest Substring Without Repeating Chars (LC #3)

Medium | Sliding Window + Set

```
int lengthOfLongestSubstring(string s) {
    unordered_set<char> window;
    int l = 0, res = 0;
    for (int r = 0; r < s.size(); r++) {
        while (window.count(s[r])) {
            window.erase(s[l]);
            l++;
        }
        window.insert(s[r]);
        res = max(res, r - l + 1);
    }
    return res;
}
```

>> Longest Repeating Character Replacement (LC #424)

Medium | *Sliding Window*

Max length substring with same char after at most k replacements. Window valid if: $\text{len} - \text{maxFreq} \leq k$.

```
int characterReplacement(string s, int k) {
    int cnt[26] = {}, l = 0, maxFreq = 0, res = 0;
    for (int r = 0; r < s.size(); r++) {
        maxFreq = max(maxFreq, ++cnt[s[r] - 'A']);
        if ((r - l + 1) - maxFreq > k) {
            cnt[s[l] - 'A']--;
            l++;
        }
        res = max(res, r - l + 1);
    }
    return res;
}
```

>> Minimum Window Substring (LC #76)

Hard | *Sliding Window + HashMap*

Find smallest substring of s containing all chars of t.

```
string minWindow(string s, string t) {
    unordered_map<char, int> need;
    for (char c : t) need[c]++;
    int missing = t.size(), l = 0, start = 0, minLen = INT_MAX;
    for (int r = 0; r < s.size(); r++) {
        if (need[s[r]] > 0) missing--;
        need[s[r]]--;
        while (missing == 0) {
            if (r - l + 1 < minLen) {
                minLen = r - l + 1;
                start = l;
            }
            need[s[l]]++;
            if (need[s[l]] > 0) missing++;
            l++;
        }
    }
    return minLen == INT_MAX ? "" : s.substr(start, minLen);
}
```

>> Minimum Size Subarray Sum (LC #209)

Medium | *Sliding Window*

Shortest subarray with sum $\geq$ target.

```
int minSubArrayLen(int target, vector<int>& nums) {
    int l = 0, sum = 0, res = INT_MAX;
    for (int r = 0; r < nums.size(); r++) {
        sum += nums[r];
        while (sum >= target) {
            res = min(res, r - l + 1);
            sum -= nums[l++];
        }
    }
    return res == INT_MAX ? 0 : res;
}
```

4. Binary Search

Core Concept

Halve search space each step -> $O(\log n)$. Works on sorted arrays or any monotonic condition. Key: identify what you are binary searching on (index, answer, etc.).

>> Binary Search Template

Easy | Foundation

```
int binarySearch(vector<int>& nums, int target) {
    int l = 0, r = nums.size() - 1;
    while (l <= r) {
        int mid = l + (r - l) / 2; // avoid overflow
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) l = mid + 1;
        else r = mid - 1;
    }
    return -1; // not found
}
```

Interview tip: Always use $l + (r - l) / 2$ instead of $(l + r) / 2$ to avoid integer overflow.

>> Search in Rotated Sorted Array (LC #33)

Medium | Binary Search

Array rotated at unknown pivot. Determine which half is sorted, check if target is in that half.

```
int search(vector<int>& nums, int target) {
    int l = 0, r = nums.size() - 1;
    while (l <= r) {
        int mid = l + (r - l) / 2;
        if (nums[mid] == target) return mid;
        if (nums[l] <= nums[mid]) { // left half sorted
            if (nums[l] <= target && target < nums[mid]) r = mid - 1;
            else l = mid + 1;
        } else { // right half sorted
            if (nums[mid] < target && target <= nums[r]) l = mid + 1;
            else r = mid - 1;
        }
    }
    return -1;
}
```

>> Find Minimum in Rotated Sorted Array (LC #153)

Medium | Binary Search

```
int findMin(vector<int>& nums) {
    int l = 0, r = nums.size() - 1;
    while (l < r) {
        int mid = l + (r - l) / 2;
        if (nums[mid] > nums[r]) l = mid + 1;
        else r = mid;
    }
    return nums[l];
}
```

>> Find First and Last Position (LC #34)

Medium | Binary Search

Use lower_bound + upper_bound pattern to find range of target.

```
vector<int> searchRange(vector<int>& nums, int target) {
    int lo = lower(nums, target), hi = lower(nums, target + 1) - 1;
    if (lo < nums.size() && nums[lo] == target) return {lo, hi};
    return {-1, -1};
}

int lower(vector<int>& nums, int target) {
    int l = 0, r = nums.size();
    while (l < r) {
        int mid = l + (r - l) / 2;
        if (nums[mid] < target) l = mid + 1;
        else r = mid;
    }
    return l;
}
```

>> Koko Eating Bananas (LC #875)

Medium | Binary Search on Answer

Classic 'binary search on answer' pattern. Search over possible eating speeds.

```
int minEatingSpeed(vector<int>& piles, int h) {
    int l = 1, r = *max_element(piles.begin(), piles.end());
    while (l < r) {
        int mid = l + (r - l) / 2;
        long hours = 0;
        for (int p : piles) hours += (p + mid - 1) / mid; // ceil div
        if (hours <= h) r = mid;
        else l = mid + 1;
    }
    return l;
}
```

>> Median of Two Sorted Arrays (LC #4)

Hard | Binary Search

Find median of two sorted arrays in $O(\log(\min(m,n)))$. Binary search on partition of smaller array.

```
double findMedianSortedArrays(vector<int>& A, vector<int>& B) {
    if (A.size() > B.size()) swap(A, B);
    int m = A.size(), n = B.size(), half = (m + n + 1) / 2;
    int l = 0, r = m;
    while (l <= r) {
        int i = (l + r) / 2, j = half - i;
        int aLeft = i > 0 ? A[i-1] : INT_MIN;
        int aRight = i < m ? A[i] : INT_MAX;
        int bLeft = j > 0 ? B[j-1] : INT_MIN;
        int bRight = j < n ? B[j] : INT_MAX;
        if (aLeft <= bRight && bLeft <= aRight) {
            if ((m + n) % 2) return max(aLeft, bLeft);
            return (max(aLeft, bLeft) + min(aRight, bRight)) / 2.0;
        } else if (aLeft > bRight) r = i - 1;
        else l = i + 1;
    }
    return 0;
}
```

5. Sorting Algorithms

Quick Summary

Algorithm	Best	Average	Worst	Space	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	Yes

Merge Sort

Divide & Conquer. Split in half, sort each, merge. Stable, $O(n \log n)$ guaranteed. Best for linked lists.

```
void mergeSort(vector<int>& arr, int l, int r) {
    if (l >= r) return;
    int mid = l + (r - l) / 2;
    mergeSort(arr, l, mid);
    mergeSort(arr, mid + 1, r);
    merge(arr, l, mid, r);
}

void merge(vector<int>& arr, int l, int mid, int r) {
    vector<int> tmp(r - l + 1);
    int i = l, j = mid + 1, k = 0;
    while (i <= mid && j <= r)
        tmp[k++] = (arr[i] <= arr[j]) ? arr[i++] : arr[j++];
    while (i <= mid) tmp[k++] = arr[i++];
    while (j <= r) tmp[k++] = arr[j++];
    for (int i = 0; i < k; i++) arr[l + i] = tmp[i];
}
```


Quick Sort

Pick pivot, partition: elements < pivot go left, > pivot go right. In-place, avg $O(n \log n)$. Worst $O(n^2)$ avoidable with random pivot.

```
void quickSort(vector<int>& arr, int lo, int hi) {
    if (lo >= hi) return;
    int pi = partition(arr, lo, hi);
    quickSort(arr, lo, pi - 1);
    quickSort(arr, pi + 1, hi);
}

int partition(vector<int>& arr, int lo, int hi) {
    int pivot = arr[hi], i = lo - 1;
    for (int j = lo; j < hi; j++) {
        if (arr[j] < pivot) swap(arr[++i], arr[j]);
    }
    swap(arr[i + 1], arr[hi]);
    return i + 1;
}
```

6. Linked Lists

Core Concepts

Singly linked list: each node has val + next pointer. Key techniques: dummy node, slow/fast pointers (Floyd's), reversing in-place, merge two lists.

Interview tip: Always consider edge cases - empty list, single node, even/odd length. Use dummy head node to simplify code when head might change.

>> Reverse Linked List (LC #206)

Easy | Iterative / Recursive

```
ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    ListNode* curr = head;
    while (curr) {
        ListNode* nxt = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nxt;
    }
    return prev;
}
```

>> Linked List Cycle (LC #141)

Easy | Floyd's Slow/Fast

```
bool hasCycle(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}
```

>> Linked List Cycle II - Find Start (LC #142)

Medium | Floyd's Algorithm

After slow & fast meet, move one pointer to head, advance both by 1 -> they meet at cycle start.

```

ListNode* detectCycle(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {
            slow = head;
            while (slow != fast) {
                slow = slow->next;
                fast = fast->next;
            }
            return slow;
        }
    }
    return nullptr;
}

```

>> Merge Two Sorted Lists (LC #21)

Easy | Dummy Node + Two Pointers

```

ListNode* mergeTwoLists(ListNode* l1, ListNode* l2) {
    ListNode dummy(0);
    ListNode* curr = &dummy;
    while (l1 && l2) {
        if (l1->val <= l2->val) { curr->next = l1; l1 = l1->next; }
        else { curr->next = l2; l2 = l2->next; }
        curr = curr->next;
    }
    curr->next = l1 ? l1 : l2;
    return dummy.next;
}

```

>> Merge K Sorted Lists (LC #23)

Hard | Min-Heap / Divide & Conquer

```

ListNode* mergeKLists(vector<ListNode*>& lists) {
    auto cmp = [](ListNode* a, ListNode* b) { return a->val > b->val; };
    priority_queue<ListNode*, vector<ListNode*>, decltype(cmp)> pq(cmp);
    for (auto* l : lists) if (l) pq.push(l);
    ListNode dummy(0);
    ListNode* curr = &dummy;
    while (!pq.empty()) {
        curr->next = pq.top(); pq.pop();
        curr = curr->next;
        if (curr->next) pq.push(curr->next);
    }
    return dummy.next;
}

```

Time: $O(N \log k)$ where N = total nodes, k = number of lists

>> Remove Nth Node From End (LC #19)

Medium | Two Pointers + Dummy

```

ListNode* removeNthFromEnd(ListNode* head, int n) {
    ListNode dummy(0, head);
    ListNode* slow = &dummy, *fast = &dummy;
    for (int i = 0; i <= n; i++) fast = fast->next;
    while (fast) { slow = slow->next; fast = fast->next; }
    ListNode* toDelete = slow->next;
    slow->next = slow->next->next;
    delete toDelete;
    return dummy.next;
}

```

>> Reorder List (LC #143)

Medium | Find Mid + Reverse + Merge

1) Find middle (slow/fast). 2) Reverse second half. 3) Merge alternating nodes.

```

void reorderList(ListNode* head) {
    if (!head || !head->next) return;
    // Find middle
    ListNode* slow = head, *fast = head->next;
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
    // Reverse second half
    ListNode* second = slow->next;
    slow->next = nullptr;
    ListNode* prev = nullptr;
    while (second) {
        ListNode* tmp = second->next;
        second->next = prev;
        prev = second;
        second = tmp;
    }
    // Merge
    ListNode* first = head;
    second = prev;
    while (second) {
        ListNode* t1 = first->next, *t2 = second->next;
        first->next = second;
        second->next = t1;
        first = t1;
        second = t2;
    }
}

```

>> Add Two Numbers (LC #2)

Medium | Linked List + Carry

Numbers stored in reverse order as linked lists. Add digit by digit with carry.

```

ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
    ListNode dummy(0);
    ListNode* curr = &dummy;
    int carry = 0;
    while (l1 || l2 || carry) {
        int sum = carry;
        if (l1) { sum += l1->val; l1 = l1->next; }
        if (l2) { sum += l2->val; l2 = l2->next; }
        carry = sum / 10;
        curr->next = new ListNode(sum % 10);
        curr = curr->next;
    }
    return dummy.next;
}

```

7. Stacks & Queues

Core Concepts

- Stack (LIFO): Push/Pop $O(1)$. Use for matching brackets, monotonic stack, undo ops, DFS. C++ `std::stack`.
- Queue (FIFO): Enqueue/Dequeue $O(1)$. Use for BFS, scheduling. C++ `std::queue`.
- Monotonic Stack: Maintains increasing/decreasing order. For next greater/smaller element problems.

>> Valid Parentheses (LC #20)

Easy | Stack

```

bool isValid(string s) {
    stack<char> st;
    unordered_map<char, char> m = {'}', '{', '}', '[', '{', '}', '[]', '{}', '{}'};
    for (char c : s) {
        if (m.count(c)) {
            if (st.empty() || st.top() != m[c]) return false;
            st.pop();
        } else {
            st.push(c);
        }
    }
    return st.empty();
}

```

>> Min Stack (LC #155)

Medium | Stack

Stack that supports push, pop, top, and getMin all in $O(1)$. Keep auxiliary min stack.

```

class MinStack {
    stack<int> st, minSt;
public:
    void push(int val) {
        st.push(val);
        int mn = minSt.empty() ? val : min(val, minSt.top());
        minSt.push(mn);
    }
    void pop() { st.pop(); minSt.pop(); }
    int top() { return st.top(); }
    int getMin() { return minSt.top(); }
};

```

>> Daily Temperatures (LC #739)

Medium | Monotonic Decreasing Stack

For each day, find how many days until warmer temperature. Push indices, pop when current > top.

```

vector<int> dailyTemperatures(vector<int>& temps) {
    int n = temps.size();
    vector<int> res(n, 0);
    stack<int> st; // indices
    for (int i = 0; i < n; i++) {
        while (!st.empty() && temps[st.top()] < temps[i]) {
            res[st.top()] = i - st.top();
            st.pop();
        }
        st.push(i);
    }
    return res;
}

```

>> Largest Rectangle in Histogram (LC #84)

Hard | Monotonic Stack

```

int largestRectangleArea(vector<int>& heights) {
    stack<int> st;
    st.push(-1);
    int maxArea = 0;
    for (int i = 0; i < heights.size(); i++) {
        while (st.top() != -1 && heights[st.top()] >= heights[i]) {
            int h = heights[st.top()]; st.pop();
            int w = i - st.top() - 1;
            maxArea = max(maxArea, h * w);
        }
        st.push(i);
    }
    while (st.top() != -1) {
        int h = heights[st.top()]; st.pop();
        int w = (int)heights.size() - st.top() - 1;
        maxArea = max(maxArea, h * w);
    }
    return maxArea;
}

```

>> Evaluate Reverse Polish Notation (LC #150)

Medium | Stack

```

int evalRPN(vector<string>& tokens) {
    stack<long long> st;
    for (auto& t : tokens) {
        if (t == "+" || t == "-" || t == "*" || t == "/") {
            long long b = st.top(); st.pop();
            long long a = st.top(); st.pop();
            if (t == "+") st.push(a + b);
            else if (t == "-") st.push(a - b);
            else if (t == "*") st.push(a * b);
            else st.push(a / b);
        } else {
            st.push(stoll(t));
        }
    }
    return st.top();
}

```

8. Trees (Binary Tree & BST)

Core Concepts

- Binary Tree: Each node has at most 2 children. Traversals: Inorder (L,Root,R), Preorder (Root,L,R), Postorder (L,R,Root), Level-order (BFS).
- BST: Left < Root < Right. Inorder gives sorted order. Search/Insert/Delete avg $O(\log n)$, worst $O(n)$.
- Key techniques: DFS (recursion/stack), BFS (queue), tree DP, height/diameter tracking.

Traversals

```
// Inorder (iterative) - gives sorted order for BST
vector<int> inorderTraversal(TreeNode* root) {
    vector<int> res;
    stack<TreeNode*> st;
    TreeNode* curr = root;
    while (curr || !st.empty()) {
        while (curr) { st.push(curr); curr = curr->left; }
        curr = st.top(); st.pop();
        res.push_back(curr->val);
        curr = curr->right;
    }
    return res;
}

// Level Order (BFS)
vector<vector<int>> levelOrder(TreeNode* root) {
    if (!root) return {};
    vector<vector<int>> res;
    queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        int sz = q.size();
        vector<int> level;
        for (int i = 0; i < sz; i++) {
            TreeNode* node = q.front(); q.pop();
            level.push_back(node->val);
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        res.push_back(level);
    }
    return res;
}
```

>> Maximum Depth of Binary Tree (LC #104)

Easy | DFS

```
int maxDepth(TreeNode* root) {
    if (!root) return 0;
    return 1 + max(maxDepth(root->left), maxDepth(root->right));
}
```

>> Invert Binary Tree (LC #226)

Easy | DFS

```
TreeNode* invertTree(TreeNode* root) {
    if (!root) return nullptr;
    swap(root->left, root->right);
    invertTree(root->left);
    invertTree(root->right);
    return root;
}
```

>> Same Tree (LC #100) / Symmetric Tree (LC #101)

Easy | DFS

```
bool isSameTree(TreeNode* p, TreeNode* q) {
    if (!p && !q) return true;
    if (!p || !q || p->val != q->val) return false;
    return isSameTree(p->left, q->left) && isSameTree(p->right, q->right);
}

// Symmetric: isMirror(root->left, root->right)
bool isMirror(TreeNode* l, TreeNode* r) {
    if (!l && !r) return true;
    if (!l || !r) return false;
    return l->val == r->val && isMirror(l->left, r->right) && isMirror(l->right, r->left);
}
```

>> Validate BST (LC #98)

Medium | DFS + Range Check

```
bool isValidBST(TreeNode* root, long lo = LONG_MIN, long hi = LONG_MAX) {
    if (!root) return true;
    if (root->val <= lo || root->val >= hi) return false;
    return isValidBST(root->left, lo, root->val)
        && isValidBST(root->right, root->val, hi);
}
```

>> Lowest Common Ancestor - Binary Tree (LC #236)

Medium | DFS

```
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
    if (!root || root == p || root == q) return root;
    TreeNode* left = lowestCommonAncestor(root->left, p, q);
    TreeNode* right = lowestCommonAncestor(root->right, p, q);
    if (left && right) return root;
    return left ? left : right;
}
```

>> Binary Tree Maximum Path Sum (LC #124)

Hard | DFS + Global Max

```
int maxPathSum(TreeNode* root) {
    int res = INT_MIN;
    function<int(TreeNode*)> dfs = [&](TreeNode* node) -> int {
        if (!node) return 0;
        int left = max(dfs(node->left), 0);
        int right = max(dfs(node->right), 0);
        res = max(res, node->val + left + right);
        return node->val + max(left, right);
    };
    dfs(root);
    return res;
}
```

>> Kth Smallest Element in BST (LC #230)

Medium | Inorder

```
int kthSmallest(TreeNode* root, int k) {
    stack<TreeNode*> st;
    TreeNode* curr = root;
    while (curr || !st.empty()) {
        while (curr) { st.push(curr); curr = curr->left; }
        curr = st.top(); st.pop();
        if (--k == 0) return curr->val;
        curr = curr->right;
    }
    return -1;
}
```

>> Construct Tree from Preorder & Inorder (LC #105)

Medium | Recursion + HashMap

```
TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
    unordered_map<int, int> idxMap;
    for (int i = 0; i < inorder.size(); i++) idxMap[inorder[i]] = i;
    int preIdx = 0;
    function<TreeNode*(int, int)> build = [&](int lo, int hi) -> TreeNode* {
        if (lo > hi) return nullptr;
        int rootVal = preorder[preIdx++];
        TreeNode* root = new TreeNode(rootVal);
        int mid = idxMap[rootVal];
        root->left = build(lo, mid - 1);
        root->right = build(mid + 1, hi);
        return root;
    };
    return build(0, inorder.size() - 1);
}
```

>> Diameter of Binary Tree (LC #543)

Easy | DFS

```
int diameterOfBinaryTree(TreeNode* root) {
    int res = 0;
    function<int(TreeNode*)> dfs = [&](TreeNode* node) -> int {
        if (!node) return 0;
        int l = dfs(node->left), r = dfs(node->right);
        res = max(res, l + r);
        return 1 + max(l, r);
    };
    dfs(root);
    return res;
}
```

>> Binary Tree Right Side View (LC #199)

Medium | BFS

```
vector<int> rightSideView(TreeNode* root) {
    if (!root) return {};
    vector<int> res;
    queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        int sz = q.size();
        for (int i = 0; i < sz; i++) {
            TreeNode* node = q.front(); q.pop();
            if (i == sz - 1) res.push_back(node->val);
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
    }
    return res;
}
```

>> Serialize and Deserialize Binary Tree (LC #297)

Hard | BFS / Preorder

Use preorder DFS with 'N' for nulls, comma-separated. Deserialize using queue/stream.

9. Heaps / Priority Queues

Core Concepts

- C++ `priority_queue<int>` = max-heap by default. For min-heap: `priority_queue<int, vector<int>, greater<int>>`.
- Top in $O(1)$, Push/Pop in $O(\log n)$. Use for Top K, median finding, merge K sorted.

>> Kth Largest Element (LC #215)

Medium | Min-Heap of size K

```
int findKthLargest(vector<int>& nums, int k) {
    priority_queue<int, vector<int>, greater<int>> minHeap;
    for (int n : nums) {
        minHeap.push(n);
        if (minHeap.size() > k) minHeap.pop();
    }
    return minHeap.top();
}
```

Time: $O(n \log k)$ | Space: $O(k)$

>> Find Median from Data Stream (LC #295)

Hard | Two Heaps

Max-heap for lower half, min-heap for upper half. Balance sizes after each insert.

```

class MedianFinder {
    priority_queue<int> lo; // max-heap
    priority_queue<int, vector<int>, greater<int>> hi; // min-heap
public:
    void addNum(int num) {
        lo.push(num);
        hi.push(lo.top()); lo.pop();
        if (hi.size() > lo.size()) {
            lo.push(hi.top()); hi.pop();
        }
    }
    double findMedian() {
        return lo.size() > hi.size() ? lo.top()
            : (lo.top() + hi.top()) / 2.0;
    }
};

```

>> Top K Frequent Words (LC #692)

Medium | Heap + HashMap

Count frequencies, use min-heap of size k. Custom comparator for tie-breaking by lexicographic order.

10. Tries

Core Concept

Trie (prefix tree) stores strings character by character. Each node has children map + isEnd flag. O(L) insert/search where L = word length. Use for autocomplete, prefix matching, word search.

>> Implement Trie (LC #208)

Medium | Trie

```

struct TrieNode {
    TrieNode* children[26] = {};
    bool isEnd = false;
};

class Trie {
    TrieNode* root;
public:
    Trie() { root = new TrieNode(); }
    void insert(string word) {
        TrieNode* node = root;
        for (char c : word) {
            int i = c - 'a';
            if (!node->children[i]) node->children[i] = new TrieNode();
            node = node->children[i];
        }
        node->isEnd = true;
    }
    bool search(string word) {
        TrieNode* node = find(word);
        return node && node->isEnd;
    }
    bool startsWith(string prefix) {
        return find(prefix) != nullptr;
    }
private:
    TrieNode* find(string& word) {
        TrieNode* node = root;
        for (char c : word) {
            int i = c - 'a';
            if (!node->children[i]) return nullptr;
            node = node->children[i];
        }
        return node;
    }
};

```

>> Word Search II (LC #212)

Hard | Trie + DFS Backtracking

Build trie from word list. DFS on board, traverse trie simultaneously. Prune branches for efficiency.

11. Graphs

Core Concepts

- Representations: Adjacency list (vector<vector<int>> or unordered_map), Adjacency matrix, Edge list.
- BFS: Queue-based, level-by-level. Shortest path in unweighted graph. $O(V+E)$.
- DFS: Stack/recursion. Cycle detection, connected components, topological sort. $O(V+E)$.
- Key algorithms: Dijkstra, Bellman-Ford, Topological Sort (Kahn's BFS), Union-Find, Kruskal/Prim MST.

BFS & DFS Templates

```
// BFS
void bfs(vector<vector<int>>& graph, int start) {
    vector<bool> visited(graph.size(), false);
    queue<int> q;
    q.push(start);
    visited[start] = true;
    while (!q.empty()) {
        int node = q.front(); q.pop();
        for (int nei : graph[node]) {
            if (!visited[nei]) {
                visited[nei] = true;
                q.push(nei);
            }
        }
    }
}

// DFS (iterative)
void dfs(vector<vector<int>>& graph, int start) {
    vector<bool> visited(graph.size(), false);
    stack<int> st;
    st.push(start);
    while (!st.empty()) {
        int node = st.top(); st.pop();
        if (visited[node]) continue;
        visited[node] = true;
        for (int nei : graph[node]) st.push(nei);
    }
}
```

>> Number of Islands (LC #200)

Medium | DFS/BFS on Grid

```
int numIslands(vector<vector<char>>& grid) {
    int m = grid.size(), n = grid[0].size(), count = 0;
    function<void(int, int)> dfs = [&](int r, int c) {
        if (r < 0 || c < 0 || r >= m || c >= n || grid[r][c] != '1') return;
        grid[r][c] = '0';
        dfs(r+1,c); dfs(r-1,c); dfs(r,c+1); dfs(r,c-1);
    };
    for (int r = 0; r < m; r++)
        for (int c = 0; c < n; c++)
            if (grid[r][c] == '1') { count++; dfs(r, c); }
    return count;
}
```

>> Clone Graph (LC #133)

Medium | BFS + HashMap

```
Node* cloneGraph(Node* node) {
    if (!node) return nullptr;
    unordered_map<Node*, Node*> clones;
    clones[node] = new Node(node->val);
    queue<Node*> q;
    q.push(node);
    while (!q.empty()) {
        Node* curr = q.front(); q.pop();
        for (Node* nei : curr->neighbors) {
            if (!clones.count(nei)) {
                clones[nei] = new Node(nei->val);
                q.push(nei);
            }
            clones[curr]->neighbors.push_back(clones[nei]);
        }
    }
    return clones[node];
}
```

>> Course Schedule / Topological Sort (LC #207)

Medium | Kahn's BFS

Detect cycle in directed graph. Use in-degree array + BFS. If all nodes processed -> no cycle.

```
bool canFinish(int n, vector<vector<int>>& prereqs) {
    vector<vector<int>> graph(n);
    vector<int> indegree(n, 0);
    for (auto& p : prereqs) {
        graph[p[1]].push_back(p[0]);
        indegree[p[0]]++;
    }
    queue<int> q;
    for (int i = 0; i < n; i++)
        if (indegree[i] == 0) q.push(i);
    int count = 0;
    while (!q.empty()) {
        int node = q.front(); q.pop();
        count++;
        for (int nei : graph[node])
            if (--indegree[nei] == 0) q.push(nei);
    }
    return count == n;
}
```

>> Course Schedule II (LC #210)

Medium | Topological Sort + Order

Same as above but return the order. Collect nodes as you process them from the queue.

>> Dijkstra's Algorithm

Medium | Shortest Path (Weighted, Non-negative)

```
vector<int> dijkstra(vector<vector<pair<int,int>>& graph, int src) {
    int n = graph.size();
    vector<int> dist(n, INT_MAX);
    dist[src] = 0;
    // {distance, node}
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
    pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;
        for (auto [v, w] : graph[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}
```

Time: $O((V+E) \log V)$ with min-heap.

>> Union-Find (Disjoint Set Union)

Medium | Foundation for Connectivity

```
class UnionFind {
    vector<int> parent, rank_;
public:
    UnionFind(int n) : parent(n), rank_(n, 0) {
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]); // path compression
        return parent[x];
    }
    bool unite(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;
        if (rank_[px] < rank_[py]) swap(px, py);
        parent[py] = px;
        if (rank_[px] == rank_[py]) rank_[px]++;
        return true;
    }
};
```

>> Surrounded Regions (LC #130)

Medium | DFS from Border

Mark border-connected 'O's. Remaining 'O's are surrounded -> flip to 'X'.

>> Word Ladder (LC #127)

Hard | BFS + Word Transformation

Shortest transformation from beginWord to endWord changing one letter at a time. BFS with word set.

>> Rotting Oranges (LC #994)

Medium | Multi-source BFS

Start BFS from all rotten oranges simultaneously. Track time (levels). Check if all fresh oranges are reached.

12. Dynamic Programming

Core Concepts

- DP = Recursion + Memoization = Optimal substructure + Overlapping subproblems.
- Patterns: 1D DP, 2D DP (grid/two sequence), Knapsack (0/1, unbounded), LIS, LCS, Interval DP, State machine.
- Steps: 1) Define state, 2) Write recurrence, 3) Base cases, 4) Determine order, 5) Optimize space.

>> Climbing Stairs (LC #70)

Easy | 1D DP / Fibonacci

```
int climbStairs(int n) {
    if (n <= 2) return n;
    int a = 1, b = 2;
    for (int i = 3; i <= n; i++) {
        int tmp = a + b;
        a = b;
        b = tmp;
    }
    return b;
}
```

>> House Robber (LC #198)

Medium | 1D DP

Can't rob adjacent houses. $dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$.

```
int rob(vector<int>& nums) {
    int n = nums.size();
    if (n == 1) return nums[0];
    int prev2 = nums[0], prev1 = max(nums[0], nums[1]);
    for (int i = 2; i < n; i++) {
        int curr = max(prev1, prev2 + nums[i]);
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}
```

>> Coin Change (LC #322)

Medium | Unbounded Knapsack

Minimum coins for amount. $dp[x] = \min \text{ over all coins of } (dp[x - \text{coin}] + 1)$.

```
int coinChange(vector<int>& coins, int amount) {
    vector<int> dp(amount + 1, amount + 1);
    dp[0] = 0;
    for (int coin : coins)
        for (int x = coin; x <= amount; x++)
            dp[x] = min(dp[x], dp[x - coin] + 1);
    return dp[amount] > amount ? -1 : dp[amount];
}
```

>> Longest Increasing Subsequence (LC #300)

Medium | DP + Binary Search

$O(n^2)$ DP or $O(n \log n)$ with patience sorting (binary search on tails array).

```
int lengthOfLIS(vector<int>& nums) {
    vector<int> tails;
    for (int n : nums) {
        auto it = lower_bound(tails.begin(), tails.end(), n);
        if (it == tails.end()) tails.push_back(n);
        else *it = n;
    }
    return tails.size();
}
```

Time: $O(n \log n)$ | Space: $O(n)$

>> Longest Common Subsequence (LC #1143)

Medium | 2D DP

```
int longestCommonSubsequence(string text1, string text2) {
    int m = text1.size(), n = text2.size();
    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++) {
            if (text1[i-1] == text2[j-1]) dp[i][j] = dp[i-1][j-1] + 1;
            else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
        }
    return dp[m][n];
}
```

>> 0/1 Knapsack

Medium | Classic DP - Frequently Asked

```
int knapsack(vector<int>& wt, vector<int>& val, int W) {
    int n = wt.size();
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < n; i++)
        for (int w = W; w >= wt[i]; w--) // reverse to avoid reuse
            dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
    return dp[W];
}
```

Time: $O(n \cdot W)$ | Space: $O(W)$ optimized

>> Word Break (LC #139)

Medium | 1D DP

```
bool wordBreak(string s, vector<string>& wordDict) {
    unordered_set<string> words(wordDict.begin(), wordDict.end());
    int n = s.size();
    vector<bool> dp(n + 1, false);
    dp[0] = true;
    for (int i = 1; i <= n; i++)
        for (int j = 0; j < i; j++)
            if (dp[j] && words.count(s.substr(j, i - j))) {
                dp[i] = true;
                break;
            }
    return dp[n];
}
```

>> Unique Paths (LC #62)

Medium | 2D DP / Math

```
int uniquePaths(int m, int n) {
    vector<int> dp(n, 1);
    for (int i = 1; i < m; i++)
        for (int j = 1; j < n; j++)
            dp[j] += dp[j - 1];
    return dp[n - 1];
}
```

>> Edit Distance (LC #72)

Medium | 2D DP

Minimum insert/delete/replace to convert word1 to word2. Classic 2D DP.

```
int minDistance(string word1, string word2) {
    int m = word1.size(), n = word2.size();
    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
    for (int i = 0; i <= m; i++) dp[i][0] = i;
    for (int j = 0; j <= n; j++) dp[0][j] = j;
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++) {
            if (word1[i-1] == word2[j-1]) dp[i][j] = dp[i-1][j-1];
            else dp[i][j] = 1 + min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]});
        }
    return dp[m][n];
}
```

>> Decode Ways (LC #91)

Medium | 1D DP

```
int numDecodings(string s) {
    int n = s.size();
    if (s[0] == '0') return 0;
    vector<int> dp(n + 1, 0);
    dp[0] = dp[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (s[i-1] != '0') dp[i] += dp[i-1];
        int two = stoi(s.substr(i-2, 2));
        if (two >= 10 && two <= 26) dp[i] += dp[i-2];
    }
    return dp[n];
}
```

>> Longest Palindromic Substring (LC #5)

Medium | Expand Around Center

```
string longestPalindrome(string s) {
    int start = 0, maxLen = 1;
    auto expand = [&](int l, int r) {
        while (l >= 0 && r < s.size() && s[l] == s[r]) { l--; r++; }
        if (r - l - 1 > maxLen) { maxLen = r - l - 1; start = l + 1; }
    };
    for (int i = 0; i < s.size(); i++) {
        expand(i, i); // odd length
        expand(i, i + 1); // even length
    }
    return s.substr(start, maxLen);
}
```

>> Maximal Square (LC #221)

Medium | 2D DP

```
int maximalSquare(vector<vector<char>>& matrix) {
    int m = matrix.size(), n = matrix[0].size(), res = 0;
    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++)
            if (matrix[i-1][j-1] == '1') {
                dp[i][j] = 1 + min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]});
                res = max(res, dp[i][j]);
            }
    return res * res;
}
```

DP Space Optimization: If $dp[i][j]$ only depends on $dp[i-1][j]$, use two variables or 1D array. E.g., Fibonacci, House Robber, Unique Paths, 0/1 Knapsack (iterate W backwards).

13. Greedy Algorithms

Core Concept

Make locally optimal choice at each step aiming for global optimum. Works when problem has greedy choice property. Common: interval scheduling, jump game, gas station.

>> Jump Game (LC #55)

Medium | Greedy

```
bool canJump(vector<int>& nums) {
    int maxReach = 0;
    for (int i = 0; i < nums.size(); i++) {
        if (i > maxReach) return false;
        maxReach = max(maxReach, i + nums[i]);
    }
    return true;
}
```

>> Jump Game II (LC #45)

Medium | Greedy / BFS

Minimum jumps to reach end. Track farthest reachable at each level.

```
int jump(vector<int>& nums) {
    int jumps = 0, curEnd = 0, farthest = 0;
    for (int i = 0; i < (int)nums.size() - 1; i++) {
        farthest = max(farthest, i + nums[i]);
        if (i == curEnd) {
            jumps++;
            curEnd = farthest;
        }
    }
    return jumps;
}
```

>> Gas Station (LC #134)

Medium | Greedy

```
int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
    int total = 0, tank = 0, start = 0;
    for (int i = 0; i < gas.size(); i++) {
        total += gas[i] - cost[i];
        tank += gas[i] - cost[i];
        if (tank < 0) {
            start = i + 1;
            tank = 0;
        }
    }
    return total >= 0 ? start : -1;
}
```

>> Non-overlapping Intervals (LC #435)

Medium | Greedy - Sort by End

Min intervals to remove for no overlap. Sort by end time, greedily keep earliest ending.

```
int eraseOverlapIntervals(vector<vector<int>>& intervals) {
    sort(intervals.begin(), intervals.end(),
        [](auto& a, auto& b) { return a[1] < b[1]; });
    int count = 0, prevEnd = INT_MIN;
    for (auto& iv : intervals) {
        if (iv[0] >= prevEnd) prevEnd = iv[1];
        else count++;
    }
    return count;
}
```

>> Candy (LC #135)

Hard | Two Pass Greedy

Each child gets at least 1 candy. Higher rating than neighbor -> more candy. Two passes: left-to-right, right-to-left.

14. Backtracking & Recursion

Core Concept

Explore all candidates by building incrementally and abandoning (backtracking) when constraint is violated. Template: choose -> explore -> un-choose.

>> Subsets (LC #78)

Medium | Backtracking

```
vector<vector<int>> subsets(vector<int>& nums) {
    vector<vector<int>> res;
    vector<int> path;
    function<void(int)> backtrack = [&](int start) {
        res.push_back(path);
        for (int i = start; i < nums.size(); i++) {
            path.push_back(nums[i]);
            backtrack(i + 1);
            path.pop_back();
        }
    };
    backtrack(0);
    return res;
}
```

>> Permutations (LC #46)

Medium | Backtracking

```
vector<vector<int>> permute(vector<int>& nums) {
    vector<vector<int>> res;
    function<void(int)> backtrack = [&](int start) {
        if (start == nums.size()) { res.push_back(nums); return; }
        for (int i = start; i < nums.size(); i++) {
            swap(nums[start], nums[i]);
            backtrack(start + 1);
            swap(nums[start], nums[i]);
        }
    };
    backtrack(0);
    return res;
}
```

>> Combination Sum (LC #39)

Medium | Backtracking

All unique combos summing to target. Each number can be used unlimited times.

```
vector<vector<int>> combinationSum(vector<int>& cand, int target) {
    sort(cand.begin(), cand.end());
    vector<vector<int>> res;
    vector<int> path;
    function<void(int, int)> bt = [&](int start, int remain) {
        if (remain == 0) { res.push_back(path); return; }
        for (int i = start; i < cand.size() && cand[i] <= remain; i++) {
            path.push_back(cand[i]);
            bt(i, remain - cand[i]); // i, not i+1: reuse allowed
            path.pop_back();
        }
    };
    bt(0, target);
    return res;
}
```

>> N-Queens (LC #51)

Hard | Backtracking

```
vector<vector<string>> solveNQueens(int n) {
    vector<vector<string>> res;
    vector<string> board(n, string(n, '.'));
    vector<bool> cols(n), diag1(2*n), diag2(2*n);
    function<void(int)> bt = [&](int r) {
        if (r == n) { res.push_back(board); return; }
        for (int c = 0; c < n; c++) {
            if (cols[c] || diag1[r+c] || diag2[r-c+n]) continue;
            board[r][c] = 'Q';
            cols[c] = diag1[r+c] = diag2[r-c+n] = true;
            bt(r + 1);
            board[r][c] = '.';
            cols[c] = diag1[r+c] = diag2[r-c+n] = false;
        }
    };
    bt(0);
    return res;
}
```

>> Word Search (LC #79)

Medium | DFS + Backtracking

```
bool exist(vector<vector<char>>& board, string word) {
    int m = board.size(), n = board[0].size();
    function<bool(int, int, int)> dfs = [&](int r, int c, int i) -> bool {
        if (i == word.size()) return true;
        if (r < 0 || c < 0 || r >= m || c >= n || board[r][c] != word[i])
            return false;
        char tmp = board[r][c];
        board[r][c] = '#';
        bool found = dfs(r+1, c, i+1) || dfs(r-1, c, i+1)
            || dfs(r, c+1, i+1) || dfs(r, c-1, i+1);
        board[r][c] = tmp;
        return found;
    };
    for (int r = 0; r < m; r++)
        for (int c = 0; c < n; c++)
            if (dfs(r, c, 0)) return true;
    return false;
}
```

>> Letter Combinations of Phone Number (LC #17)

Medium | Backtracking

```
vector<string> letterCombinations(string digits) {
    if (digits.empty()) return {};
    vector<string> map = {"", "", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"};
    vector<string> res;
    string path;
    function<void(int)> bt = [&](int idx) {
        if (idx == digits.size()) { res.push_back(path); return; }
        for (char c : map[digits[idx] - '0']) {
            path.push_back(c);
            bt(idx + 1);
            path.pop_back();
        }
    };
    bt(0);
    return res;
}
```

>> Generate Parentheses (LC #22)

Medium | Backtracking

```
vector<string> generateParenthesis(int n) {
    vector<string> res;
    string path;
    function<void(int, int)> bt = [&](int open, int close) {
        if (path.size() == 2 * n) { res.push_back(path); return; }
        if (open < n) { path += '('; bt(open+1, close); path.pop_back(); }
        if (close < open) { path += ')'; bt(open, close+1); path.pop_back(); }
    };
    bt(0, 0);
    return res;
}
```

15. Bit Manipulation

Core Operations

- AND (&): Both bits 1 -> 1. Check even/odd: $n \& 1$.
- OR (|): Either bit 1 -> 1. Set a bit: $n \mid (1 \ll k)$.
- XOR (^): Different bits -> 1. $a \wedge a = 0$, $a \wedge 0 = a$. Find single number, swap without temp.
- Left shift (<<): Multiply by 2. Right shift (>>): Divide by 2.
- $n \& (n-1)$: Removes lowest set bit. Power of 2 check, count set bits.

>> Single Number (LC #136)

Easy | XOR

```
int singleNumber(vector<int>& nums) {
    int res = 0;
    for (int n : nums) res ^= n;
    return res;
}
```

>> Number of 1 Bits (LC #191)

Easy | Brian Kernighan's

```
int hammingWeight(uint32_t n) {
    int count = 0;
    while (n) {
        n &= (n - 1); // clear lowest set bit
        count++;
    }
    return count;
}
```

>> Missing Number (LC #268)

Easy | XOR / Math

```
int missingNumber(vector<int>& nums) {
    int n = nums.size(), res = n;
    for (int i = 0; i < n; i++) res ^= i ^ nums[i];
    return res;
}
```


>> Counting Bits (LC #338)

Easy | DP + Bit

```
vector<int> countBits(int n) {
    vector<int> dp(n + 1);
    for (int i = 1; i <= n; i++)
        dp[i] = dp[i >> 1] + (i & 1);
    return dp;
}
```

>> Reverse Bits (LC #190)

Easy | Bit Manipulation

```
uint32_t reverseBits(uint32_t n) {
    uint32_t res = 0;
    for (int i = 0; i < 32; i++) {
        res = (res << 1) | (n & 1);
        n >>= 1;
    }
    return res;
}
```

16. Intervals

Core Pattern

Sort intervals by start (or end). Process sequentially checking overlaps. Two intervals [a,b] and [c,d] overlap if $a < d$ and $c < b$.

>> Merge Intervals (LC #56)

Medium | Sort + Merge

```
vector<vector<int>> merge(vector<vector<int>>& intervals) {
    sort(intervals.begin(), intervals.end());
    vector<vector<int>> merged = {intervals[0]};
    for (int i = 1; i < intervals.size(); i++) {
        if (intervals[i][0] <= merged.back()[1])
            merged.back()[1] = max(merged.back()[1], intervals[i][1]);
        else
            merged.push_back(intervals[i]);
    }
    return merged;
}
```

>> Insert Interval (LC #57)

Medium | Linear Scan + Merge

```
vector<vector<int>> insert(vector<vector<int>>& intervals,
                        vector<int>& newInterval) {
    vector<vector<int>> res;
    for (int i = 0; i < intervals.size(); i++) {
        if (intervals[i][1] < newInterval[0]) {
            res.push_back(intervals[i]);
        } else if (intervals[i][0] > newInterval[1]) {
            res.push_back(newInterval);
            res.insert(res.end(), intervals.begin() + i, intervals.end());
            return res;
        } else {
            newInterval[0] = min(newInterval[0], intervals[i][0]);
            newInterval[1] = max(newInterval[1], intervals[i][1]);
        }
    }
    res.push_back(newInterval);
    return res;
}
```

>> Min Arrows to Burst Balloons (LC #452)

Medium | Greedy - Sort by End

Sort by end coordinate. Greedily shoot at end of first unpoped balloon. Count arrows.

17. Math & Matrix

>> Rotate Image 90 deg (LC #48)

Medium | Matrix - Transpose + Reverse

Transpose matrix, then reverse each row = 90 degree clockwise rotation.

```
void rotate(vector<vector<int>>& matrix) {
    int n = matrix.size();
    for (int i = 0; i < n; i++)
        for (int j = i; j < n; j++)
            swap(matrix[i][j], matrix[j][i]);
    for (auto& row : matrix)
        reverse(row.begin(), row.end());
}
```

>> Spiral Matrix (LC #54)

Medium | Simulation

```
vector<int> spiralOrder(vector<vector<int>>& matrix) {
    vector<int> res;
    int top = 0, bot = matrix.size()-1, left = 0, right = matrix[0].size()-1;
    while (top <= bot && left <= right) {
        for (int c = left; c <= right; c++) res.push_back(matrix[top][c]);
        top++;
        for (int r = top; r <= bot; r++) res.push_back(matrix[r][right]);
        right--;
        if (top <= bot)
            for (int c = right; c >= left; c--) res.push_back(matrix[bot][c]);
        bot--;
        if (left <= right)
            for (int r = bot; r >= top; r--) res.push_back(matrix[r][left]);
        left++;
    }
    return res;
}
```

>> Pow(x, n) - Fast Exponentiation (LC #50)

Medium | Binary Exponentiation

```
double myPow(double x, int n) {
    long long p = abs((long long)n);
    double res = 1.0;
    while (p) {
        if (p & 1) res *= x;
        x *= x;
        p >>= 1;
    }
    return n < 0 ? 1.0 / res : res;
}
```

>> Next Permutation (LC #31)

Medium | Array / Math

1) Find largest i where nums[i] < nums[i+1]. 2) Find largest j > i where nums[j] > nums[i]. 3) Swap. 4) Reverse from i+1.

```
void nextPermutation(vector<int>& nums) {
    int i = nums.size() - 2;
    while (i >= 0 && nums[i] >= nums[i+1]) i--;
    if (i >= 0) {
        int j = nums.size() - 1;
        while (nums[j] <= nums[i]) j--;
        swap(nums[i], nums[j]);
    }
    reverse(nums.begin() + i + 1, nums.end());
}
```

>> Sieve of Eratosthenes (Count Primes LC #204)

Medium | Math

```
int countPrimes(int n) {
    if (n <= 2) return 0;
    vector<bool> isPrime(n, true);
    isPrime[0] = isPrime[1] = false;
    for (int i = 2; i * i < n; i++)
        if (isPrime[i])
            for (int j = i * i; j < n; j += i)
                isPrime[j] = false;
    return count(isPrime.begin(), isPrime.end(), true);
}
```

18. Design Problems

Commonly Asked in Interviews

>> LRU Cache (LC #146)

Medium | Hash Map + Doubly Linked List

get(key) and put(key, val) in O(1). Use unordered_map + doubly linked list (std::list). Map stores iterators to list nodes.

```
class LRUCache {
    int cap;
    list<pair<int,int>> dll; // {key, val}, front = most recent
    unordered_map<int, list<pair<int,int>>::iterator> mp;
public:
    LRUCache(int capacity) : cap(capacity) {}
    int get(int key) {
        if (!mp.count(key)) return -1;
        dll.splice(dll.begin(), dll, mp[key]); // move to front
        return mp[key]->second;
    }
    void put(int key, int value) {
        if (mp.count(key)) {
            mp[key]->second = value;
            dll.splice(dll.begin(), dll, mp[key]);
        } else {
            if (dll.size() == cap) {
                mp.erase(dll.back().first);
                dll.pop_back();
            }
            dll.emplace_front(key, value);
            mp[key] = dll.begin();
        }
    }
};
```

>> Insert Delete GetRandom O(1) (LC #380)

Medium | HashMap + Vector

Use vector for O(1) random access and hashmap {val -> index} for O(1) insert/delete. On remove, swap with last element.

```
class RandomizedSet {
    vector<int> nums;
    unordered_map<int, int> idx;
public:
    bool insert(int val) {
        if (idx.count(val)) return false;
        idx[val] = nums.size();
        nums.push_back(val);
        return true;
    }
    bool remove(int val) {
        if (!idx.count(val)) return false;
        int last = nums.back();
        nums[idx[val]] = last;
        idx[last] = idx[val];
        nums.pop_back();
        idx.erase(val);
        return true;
    }
    int getRandom() { return nums[rand() % nums.size()]; }
};
```

19. Complexity Cheat Sheet & Pattern Map

Time Complexity Hierarchy

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

Data Structure Operations

Data Structure	Access	Search	Insert	Delete	Space
Array / vector	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Linked List	$O(n)$	$O(n)$	$O(1)^*$	$O(1)^*$	$O(n)$
Hash Table (unordered_map)	N/A	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	$O(n)$
Stack / Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
BST (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Heap (priority_queue)	$O(1)$ top	$O(n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Trie	N/A	$O(L)$	$O(L)$	$O(L)$	$O(26 * L * N)$

* $O(1)$ insert/delete at known position. Finding the position first takes $O(n)$.

Algorithm Complexities

Algorithm	Time	Space	Notes
Binary Search	$O(\log n)$	$O(1)$	Sorted input required
BFS / DFS	$O(V + E)$	$O(V)$	Graph traversal
Dijkstra (heap)	$O((V+E) \log V)$	$O(V)$	Non-negative weights
Merge Sort	$O(n \log n)$	$O(n)$	Stable, guaranteed
Quick Sort	$O(n \log n)$ avg	$O(\log n)$	In-place, not stable
Kadane's	$O(n)$	$O(1)$	Max subarray sum
Topological Sort	$O(V + E)$	$O(V)$	DAG only (Kahn's BFS)
Union-Find	$\sim O(1)$ amort.	$O(n)$	Path compression + rank
Knapsack 0/1	$O(n * W)$	$O(W)$	DP, pseudo-polynomial
LIS (bisect)	$O(n \log n)$	$O(n)$	Patience sorting

Problem Pattern -> Technique Map

If you see...	Think of...
Sorted array / search space	Binary Search, Two Pointers
Top/Min/Max K elements	Heap (priority_queue)
All permutations/subsets/combos	Backtracking
Connected components	BFS, DFS, Union-Find
Shortest path (unweighted)	BFS (level-order)
Shortest path (weighted)	Dijkstra, Bellman-Ford
Dependency ordering	Topological Sort (Kahn's)
Overlapping subproblems	DP (memo or tabulation)
Max/min subarray or window	Kadane's / Sliding Window
String matching / prefix	Sliding Window, Trie, KMP
Tree path or sum problems	DFS recursion, Tree DP
Parentheses / nesting / undo	Stack
Next greater / smaller element	Monotonic Stack
Range queries / prefix sums	Prefix Sum, Segment Tree, BIT
Interval merge / overlap	Sort + Greedy
Find single / missing element	XOR, Math
$O(1)$ design + random access	HashMap + Vector (LC #380)
$O(1)$ cache with eviction	HashMap + Doubly Linked List (LRU)

C++ STL Quick Reference for Interviews

Container / Algo	Key Methods / Usage
vector<T>	push_back, pop_back, size, [], begin/end, resize, sort(v.begin(), v.end())
unordered_map<K,V>	mp[key], mp.count(key), mp.erase(key), for(auto& [k,v] : mp)
unordered_set<T>	insert, count, erase, find
stack<T>	push, pop, top, empty, size
queue<T>	push, pop, front, empty, size
priority_queue<T>	push, pop, top (max-heap). Min-heap: priority_queue<T,vector<T>,greater<T>>
string	substr(pos,len), find, push_back, pop_back, size, s[i], stoi, to_string
sort / lower_bound	sort(b,e), sort(b,e,cmp), lower_bound(b,e,val), upper_bound
list<T>	Doubly linked list. splice, push_front/back. Used in LRU Cache.

Interview Tips: 1) Clarify constraints & edge cases first. 2) Start brute force, then optimize. 3) Think out loud. 4) Test with small examples. 5) State time/space complexity. 6) Ask: Can sorting help? Can a hash map reduce lookup? Can two pointers work? 7) C++ specific: use 'long long' for overflow, use 'auto' and structured bindings for cleaner code.